



Application Help for Manage PODs 2.0

Generated on: 2026-04-15 12:00:22 GMT+0000

|

Confidential

Original content: <https://help.sap.com/docs/help/95abdf318cec40bb84bc487fdaa03691?locale=en-US&state=PRODUCTION&version=2511>

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit <https://help.sap.com/docs/disclaimer>.

POD 2.0 Plugin Developer's Guide

This guide is intended to assist you in creating plugins for use with Production Operator Dashboards.

In this guide, you'll learn how to develop a custom POD plugin for use with SAP Digital Manufacturing. For additional information and sample plugins, see [POD 2.0 Extensions](#) ➡

Overview

Learn about the types of plugins you can develop.

A plugin is a UI5 module that enhances the functionality of a Production Operator Dashboard (POD). SAP provides a number of core plugins that are available to use in your POD right away. You can extend these plugins to make minor modifications, or develop your own custom plugin completely from scratch.

There are two types of POD plugins:

- **Widgets** are configurable visual elements that can be added to the POD UI, and are the primary means of presenting information or receiving input from the operator.
- **Actions** are configurable procedures that occur in response to user input. A sequence of actions can be configured to run for each event supported by a widget.

→ Remember

UI5 updates can impact the plugins and extensions that you develop. Since UI5 is updated with every release of SAP Digital Manufacturing for execution, it's a best practice to test the plugins you've developed for every release of SAP Digital Manufacturing for execution.

You can create and extend multiple plugins of various types and package them into a single project called an extension. Once your extension has been uploaded to the [Manage PODs 2.0](#) app, your plugins are available for use in a POD as well as the [Design Production Processes](#) app. Continue reading to understand how to customize POD plugins to best fit your needs.

Plugin Lifecycle

Learn how widgets and actions behave throughout their lifecycles.

Plugin lifecycle refers to the full sequence of stages a widget or action goes through during its existence in a POD. Understanding their lifecycles helps you develop responsive, efficient, and predictable plugins.

It's important to know that in design-mode, all plugins in your POD configuration are instantiated across all pages and dialogs. Whereas in the POD runtime, only the plugins that are present on the active page or open dialog are loaded.

Widget Lifecycle

Below are the possible steps in a widget lifecycle and when they occur:

	Initial Startup	Page Navigate (forward)	Page Navigate (back)	Dialog Open	Dialog Close
The Page Leave and Dialog Close events are executed, along with any configured actions		☑	☑		☑
onExit() is called for each widget in the page or dialog			☑		☑
The page or dialog is destroyed, along with all widget view controls			☑		☑
The page or dialog widget and any child widgets are instantiated and _createView() is called i Note The _createView() function may be asynchronous and is awaited. Asynchronous _createView calls are parallelized.	☑	☑		☑	
onInit() is called for the page or dialog widget and any child widgets	☑	☑		☑	
The Page Enter and Dialog Open events are executed, along with any configured actions	☑	☑	☑	☑	

Action Lifecycle

Actions are executed in the order they are defined within a widget event. A widget can have any number of events, and each of a widget's events may have one or more actions configured. When a widget is instantiated, an `EventHandler` is automatically created for each of its supported event types. When a user triggers one of these events (such as by clicking a button), the following process occurs:

1. The `EventHandler` is invoked with the relevant SAPUI5 `sap.ui.base.Event` object.
2. Each action in the configured sequence is instantiated.

3. If an action includes an `onInit()` method, it is called.

i Note

This call can be a synchronous or asynchronous. If it is asynchronous, then `EventHandler` will await the `onInit()` method to complete.

i Note

If an action throws an error during its `onInit()` step, the sequence is immediately halted. No further actions will be initialized, and no actions will be executed.

4. An `ActionContext` is created and passed to each action's `execute()` function.

5. Each action's `execute()` method is called and awaited in sequence.

i Note

If an action calls `ActionContext.abort()` or throws an error, the sequence is immediately halted.

6. Once the sequence ends, any action that defines `onExit()` will have it called and awaited sequentially.

Every action can result in one of three outcomes:

- **Success:** `execute()` completes normally or its promise resolves.
- **Action Sequence Aborted:** A soft exit (such as the user canceling a dialog) where `abort()` is explicitly called.
- **Error:** An error is thrown or the promise is rejected.

POD Context

Learn how to manage states within POD Context.

POD Context refers to the wrapper around an `SAPUI5 JSONModel` which provides enhanced observer pattern capabilities. It allows plugins to easily access and react to real-time POD data without manual change tracking or complex event handling.

POD States

Effectively managing states in your POD helps keep your components organized, modular, and easy to maintain. Within the POD Context, there are two types of state you'll work with: internal and shared. Both serve a different purpose and follow specific guidelines to ensure your code remains loosely coupled and easy to extend.

Internal State

Internal state refers to data that belongs exclusively to a single component, such as a widget, an action, or a related helper class. This might include arrays, objects, or other properties used within that component. Internal state should be self-contained and not rely on or directly affect other components.

If you're binding internal state to a model, it is recommended to use a model named `internal`. Avoid using unnamed models when assigning data to controls, since this can lead to confusion and potential conflicts.

Example

If you're building a selection dialog that lists work centers, you might use an internal array to hold the list of work centers. This data only exists within the dialog. Once a user makes a selection, only the result is passed on to other parts of the application. Although shared data like work centers should follow consistent structures, internal data can use whatever structure makes sense for your component.

Shared State

Shared state is used when multiple components need to access or update the same data. This state is stored in the `POD Context` model using clear and consistent data structures.

Instead of calling functions on other components to pass data around, you can update a specific path in the `POD Context` model. Other components can then bind to that path and respond automatically when the data changes, keeping your components independent from one another and easier to manage.

❖ Example

Imagine you have a widget that displays operation activities based on a selected work list item. The widget doesn't need to know where the selection came from, it just binds to `ModelPath.SelectedWorkListItems` in the `POD Context` model and waits for the data to be available. A plugin such as the [Work List Filter Bar](#) plugin can update this path. This way, neither component needs to know what the other one is or where it appears in the `POD` layout.

Observing Changes

Changes to shared state can be observed by subscribing to the respective model path in the `POD Context`. If one path is subscribed, the provided callback function is called each time the respective value changes. If multiple paths are subscribed, the callback is invoked each time any of the paths' values change. We recommend that you track changes in the subscription callback since a new object or array may still represent the same entity or entities.

i Note

It is important to unsubscribe when the subscribing component is destroyed or otherwise no longer needs to consume the shared data.

Most components need to check the current state of a path with `PodContext.get` and then subscribe to future changes with `PodContext.subscribe`. Components that only require the current value can use the `PodContext.getWhenAvailable` function to get the first available value, even if it has not been set yet, without subscribing to future changes.

Additional Functionality

A `PodContextObject` instance can be constructed using a plain object with the same properties as are specified in its metadata. Properties that do not exist in the class's static metadata will be ignored.

Because an instance of the class is also an object with the same properties, the constructor can be used to easily create a shallow clone. This is helpful to avoid modifying shared data by reference. Changes to shared state should be communicated by setting a new or cloned object to the `POD Context`, not by modifying the current shared object in-place.

≡ Sample Code

```
const oWorkCenter = new WorkCenter({
  workCenter: "MIXING",
  description: "Ink Mixing"
});
```

```
const oClone = new WorkCenter(oWorkCenter);
```

Set Up Your Project

Organize your project based on the recommended structure.

Context

Before starting any plugin development, it's important to strategically plan the structure of your extension to best organize your code. Complete the following steps to set up your project in accordance with our recommendations.

i Note

While you can structure your extension however you like, the examples and sample code throughout this guide reflect the recommended project set up on this page.

Procedure

1. Create the following three folders in your working directory:

- `action` for custom actions
- `widget` for custom widgets
- `util` for reusable logic and functions

→ Tip

Simple extensions containing one plugin can have only one folder, or no folders if the JavaScript files are located directly in the root of the project.

Complex extensions can have more folders or nested namespaces to better organize the various modules.

i Note

Depending on the complexity of your extension, you may also want to create folders for property editors, custom SAPUI5 controls, or utility classes.

2. Create an empty `extension.json` file at the root of your project.

i Note

All custom extensions must have an `extension.json` file at the root of the project. This file indicates the location each widget or action can be loaded from, and their unique type identifier used in the POD configuration.

3. Choose a unique namespace prefix for your project. It can include your company or team name.

❖ Example

Based on the above guidance, a valid namespace could be `myCompany/extension`. The module path for a plugin called `MyWidget` in this project would be `myCompany/extension/widget/MyWidget`.

These examples will be used throughout the rest of this guide.

Results

You've set up your project in accordance with best practices and familiarized yourself with what to expect in the sample code throughout the rest of this guide.

Develop a Widget

Learn how to develop a custom widget.

Widgets are configurable visual elements that can be added to a POD's interface, and are the primary means of presenting information or receiving input from the operator. A widget typically offers one or more properties that can be used to modify the behavior or appearance of the widget, and may expose events indicated by user interactions that can result in a configured sequence of actions being executed. Widgets can also be containers for other widgets.

Continue reading to learn how to develop your own widget.

Create a Widget Class

Create a custom JavaScript class for your widget.

Context

When developing a new widget, start by creating a JavaScript class that extends the appropriate base widget. This page outlines the steps and naming conventions required to create a widget class.

Procedure

1. In the widget folder of your project, create a new JavaScript file with the desired name for your widget. For example, `MyWidget.js`.
2. Within the file, use [sap.ui.define](#) to create a new SAPUI5 module which imports the `Widget` parent class and exports a class which inherits from it.

Sample Code

```
sap.ui.define([
    "sap/dm/dme/pod2/widget/Widget"
],
(
    Widget
) => {
    "use strict";

    class MyWidget extends Widget {
    }

    return MyWidget;
});
```

Add Widget Metadata

Add metadata to your widget class.

Context

Providing custom metadata is necessary to define how your widget is presented and categorized in the POD editor interface. This is done by implementing a set of static methods in your widget class. These methods inform the editor about the widget's name, icon, category, and optional help documentation.

Procedure

Add the following static methods to your widget class definition:

- a. `getDisplayName()`: Specifies the name shown for the widget in the POD editor plugin list. This should return a localized string. Refer to [Internationalization](#) for localization guidelines.
- b. `getDescription()` (optional): Specifies the tooltip shown for the widget in the POD editor plugin list. This should return a localized string. Refer to [Internationalization](#) for localization guidelines.
- c. `getIcon()`: Returns the icon displayed alongside the widget in the palette. If omitted, a default icon (question mark) will be used. Browse available icons in the [UI5 Icon Explorer](#) .
- d. `getCategory()`: Defines the widget category grouping in the POD editor. Any localized string can be returned, and it is common to create a dedicated category for all custom extensions. If not overridden, the widget appears in the Uncategorized group.
- e. `getHelpUrl()` (optional): Provides a link to widget documentation. If implemented, a help icon appears in the widget's Properties panel. Clicking it will open the URL in a new browser tab. If not implemented, no help icon is shown.

Sample Code

```
class MyWidget extends Widget {
    /**
     * Returns the display name shown in the widget palette.
     * @override
     * @returns {string}
     */
    static getDisplayName() {
        return "Sample Widget";
    }

    /**
     * (Optional) Returns the display name shown in the widget palette.
     * @override
     * @returns {string}
     */
    static getDescription() {
        return "A sample widget demonstrating the process of creating an extension.";
    }

    /**
     * Returns the icon shown in the widget palette.
     * @override
     * @returns {string}
     */
    static getIcon() {
```



```

        return "sap-icon://filter";
    }

    /**
     * Returns the category the widget belongs to in the POD Designer.
     * @override
     * @returns {string}
     */
    static getCategory() {
        return "Custom Widgets";
    }

    /**
     * (Optional) Returns a URL to help documentation for the widget.
     * @override
     * @returns {string}
     */
    static getHelpUrl() {
        return "https://help.sap.com/docs/sap-digital-manufacturing/widgets/sample-widget";
    }
}

```

Create Widget UI

Configure how your widget appears to the user.

Context

When developing a widget, it's necessary to define the user interface using a `_createView()` method. You can configure this directly in JavaScript or load code from an SAPUI5 XML view or fragment.

i Note

We recommend using the JavaScript approach, as it is faster and more flexible. This is the method documented below.

Procedure

1. Define a method named `_createView()` inside your widget class definition.
2. Within your `_createView()` method, return a SAPUI5 control (any UI5 object from [sap.ui.core.Element](#)) that acts as the view for your widget. This view can be a single control like a `FilterBar`, or it can be a layout control like an `HBox` or `VBox` that holds a hierarchy of controls.

Example

In this example, the `MyWidget` class uses an `sap.m.Panel` as its main control, and the panel instance is given the same element ID as the widget's unique ID. Inside the panel is a single `sap.m.Label` and `sap.m.Input` as a simple demonstration.

i Note

Additional dependencies have been imported at the top of the file for the necessary SAPUI5 controls. The static metadata functions have not been removed from the class, but have been hidden from the example code below for brevity.

i Note

The `_createView` method can be asynchronous, for example if a view fragment needs to be loaded from a separate file.

Sample Code

```
sap.ui.define([
    "sap/m/Input",
    "sap/m/Label",
    "sap/m/Panel",
    "sap/dm/dme/pod2/widget/Widget"
],
(
    Input,
    Label,
    Panel,
    Widget
) => {
    "use strict";

    class MyWidget extends Widget {
        /**
         * Create a panel to contain all the widget's UI elements.
         *
         * @override
         * @returns {sap.m.Panel}
         */
        _createView() {
            // Instantiate child controls of the widget's main view.
            const oInput = new Input();
            const oLabel = new Label({
                labelFor: oInput,
                showColon: true,
                text: "My Input"
            });

            // Instantiate the panel with the same ID as the widget, set its default properties
            return new Panel(this.getId(), {
                headerText: "My Custom Panel Plugin",
                content: [
                    oLabel,
                    oInput
                ]
            });
        }
    }

    return MyWidget;
});
```

Enable Drag and Drop

Ensure your widget can be dragged and dropped into the Manage PODs 2.0 editor.

Context

Enabling drag and drop is essential to ensure your widget is compatible with the [Manage PODs 2.0](#) editor.

Some SAPUI5 controls support drag and drop without additional configuration. If your control does not support drag and drop, the editor will display warnings similar to the following:

```
2024-09-28T01:39:19.467Z [WARN] sap.dm.dme.pod2.designer.PreviewPanel: Control sap.ui.comp.filterba
2024-09-28T01:39:19.467Z [WARN] sap.dm.dme.pod2.designer.PreviewPanel: Control sap.ui.comp.filterba
```

i Note

If your plugin is inserted with no warnings, continue to [Add Widget Properties](#). If you receive one or both of the above warnings, follow the procedure below.

Procedure

1. Create a folder to house your control files if one does not already exist. In this folder, create a file named `Custom<Control>.js` where `<Control>` is the name of the SAPUI5 control you are overriding.
2. Define a class named `Custom<Control>`.
3. Subclass the desired SAPUI5 control class and add metadata to enable drag and drop support.

Sample Code

The control class in this example is named `CustomIcon`.

```
sap.ui.define([
    "sap/ui/core/Icon",
],
/**
 * @param {typeof sap.ui.core.Icon} Icon
 */
(
    Icon
) => {
    "use strict";

    /**
     * Overrides the standard Icon control to enable drag and drop in the POD Designer.
     *
     * @class
     * @alias sap.dm.dme.pod2.control.CustomIcon
     * @extends sap.ui.core.Icon
     */
    // @ts-ignore
    let CustomIcon = Icon.extend("sap.dm.dme.pod2.control.CustomIcon", {
        metadata: {
            // Enable drag and drop for this control in POD Designer
            dnd: {
                draggable: true,
                droppable: true
            }
        },
        renderer: {}, // Disable custom renderer loading
    });

    return CustomIcon;
});
```

Add Widget Properties

Define configurable properties for your widget.

Context

Widgets can include properties that allow you to customize how the widget looks and behaves. The **Properties** panel in the **Manage PODs 2.0** editor displays all the properties supported by a widget and their values. Widget properties can have different controls depending on the data type of the property. For example, text box, checkbox, or drop-down list.

Procedure

1. In your widget's JavaScript file, add a `getProperties` method to your widget class and return an array of `WidgetProperty` objects. Each object describes one configurable property:
 - a. `displayName`: The text to display in the Properties panel for this property.
 - b. `description`: The tooltip to display when the user mouses over the property to give more detailed information on what the property does.
 - c. `category`: The widget properties shown in the Properties panel can be grouped together with similar properties.
 - d. `propertyEditor`: An instance of `sap.dm.dme.pod2.propertyeditor.PropertyEditor` that provides a control to display and modify the widget property. Typically you would use an existing property editor class. For more information, refer to [Property Editors](#).

Sample Code

```
sap.ui.define([
    "sap/dm/dme/pod2/widget/Widget",
    "sap/dm/dme/pod2/widget/metadata/WidgetProperty",
    "sap/dm/dme/pod2/propertyeditor/BooleanPropertyEditor"
],
/**
 * @param {typeof sap.dm.dme.pod2.widget.Widget} Widget
 * @param {typeof sap.dm.dme.pod2.widget.metadata.WidgetProperty} WidgetProperty
 * @param {typeof sap.dm.dme.pod2.propertyeditor.BooleanPropertyEditor} BooleanPropertyEditor
 */
(
    Widget,
    WidgetProperty,
    BooleanPropertyEditor
) => {
    "use strict";

    class <class> extends Widget {
        getProperties() {
            return [
                new WidgetProperty({
                    displayName: "SFC Visible",
                    description: "Displays the SFC field.",
                    category: "SFC",
                    propertyEditor: new BooleanPropertyEditor(this, "sfcVisible")
                }),
                ...
            ];
        }
    }

    return <class>;
});
```

i Note

The `getProperties` example above uses a hard coded display name and description in English. To enable support in other languages, the widget should use the `getI18nText` method provided by the `Widget` base class to get a translated string. See [Internationalization](#) for details.

2. Add default values for each property to the result of the `getDefaultConfig` static method.

Sample Code

```
static getDefaultConfig() {
    return {
        properties: {
            sfcVisible: true
        }
    };
}
```

Note

Widget property values can be of any data type. Each property made available in `getProperties` should be interpreted according to the expected data type, and an appropriate default value should be present in the default configuration returned by `getDefaultConfig`.

3. If changing a widget property in the editor of the **Manage PODs 2.0** app also affects its appearance, the widget should override the `setPropertyValue` method to ensure the widget's appearance matches the configured value:

Sample Code

```
/**
 * Override the setProperty to handle any properties that need to update the control.
 *
 * @override
 * @property {string} sName The name of the property
 * @property {any} vValue The value of the property
 */
setProperty(sName, vValue) {
    if (sName === "color") {
        ...
    } else if (sName === "icon") {
        ...
    }

    // Call super method to update the configuration
    super.setProperty(sName, vValue);
}
```

Next Steps

To retrieve the value of a property you've configured, call `getPropertyValue`:

```
/**
 * @override
 * @param {sap.dm.dme.pod2.widget.WidgetConfig} oConfig
 */
constructor(oConfig) {
    super(oConfig);

    if (this.getPropertyValue("sfcVisible")) {
        ...
    }
}
```

Property Examples

Below are examples of how you can customize your widget using properties.

Add a Configurable Property

This example adds a new property to set the visibility of the label created during `_createView`. Note that the configured value is used for the label's visibility property when it is instantiated.

Sample Code

```
sap.ui.define([
    "sap/m/Input",
    "sap/m/Label",
    "sap/m/Panel",
    "sap/dm/dme/pod2/propertyeditor/BooleanPropertyEditor",
    "sap/dm/dme/pod2/propertyeditor/PropertyCategory",
    "sap/dm/dme/pod2/widget/Widget",
    "sap/dm/dme/pod2/widget/metadata/WidgetProperty"
],
(
    Input,
    Label,
    Panel,
    BooleanPropertyEditor,
    PropertyCategory,
    Widget,
    WidgetProperty
) => {
    "use strict";

    class MyWidget extends Widget {
        /**
         * Create a panel to contain all the widget's UI elements.
         *
         * @override
         * @returns {sap.m.Panel}
         */
        _createView() {
            const oInput = new Input();
            const oLabel = new Label({
                labelFor: oInput,
                showColon: true,
                text: "My Input",
                visible: this.getPropertyValue("labelVisible")
            });

            return new Panel({
                headerText: "My Custom Panel Widget",
                content: [
                    oLabel,
                    oInput
                ]
            });
        }

        /**
         * @override
         * @returns {Array<sap.dm.dme.pod2.widget.metadata.WidgetProperty>}
         */
        getProperties() {
            return [
                new WidgetProperty({
                    displayName: "Label Visible",
                    description: "Displays the label",
                    category: PropertyCategory.Main,
                    propertyEditor: new BooleanPropertyEditor(this, "labelVisible")
                })
            ];
        }
    }

    return MyWidget;
});
```

Assign Default Values for Unset Properties

While this will allow the user to choose the visibility of the label, if no value has been set its initial default of `undefined` (absent from the configuration) will result in an invisible label. This should be addressed in two ways:

1. Set an initial value to the configuration when the widget is added to a POD.
2. Ensure the default value is correct if the property is missing from the configuration. Keep in mind that new properties added in future versions of a widget will not be present in existing saved configurations for that widget.

To account for this, you can implement a static `getDefaultConfig` method and override `getPropertyValue` to coalesce the default in cases where it is absent.

i Note

In this example the `getDefaultConfig` and `getPropertyValue` methods are shown. Other methods from previous examples are hidden for brevity, but are still present in the class.

Sample Code

```
sap.ui.define([
    "sap/m/Input",
    "sap/m/Label",
    "sap/m/Panel",
    "sap/dm/dme/pod2/propertyeditor/BooleanPropertyEditor",
    "sap/dm/dme/pod2/propertyeditor/PropertyCategory",
    "sap/dm/dme/pod2/widget/Widget",
    "sap/dm/dme/pod2/widget/metadata/WidgetProperty"
],
(
    Input,
    Label,
    Panel,
    BooleanPropertyEditor,
    PropertyCategory,
    Widget,
    WidgetProperty
) => {
    "use strict";

    class MyWidget extends Widget {
        /**
         * @override
         * @returns {sap.dm.dme.pod2.widget.DefaultWidgetConfig}
         */
        getDefaultConfig() {
            return {
                properties: {
                    "labelVisible": true
                }
            };
        }

        /**
         * Override to coalesce default values for absent properties.
         */
        /**
         * @override
         * @param {string} sName
         * @returns {any}
         */
        /**
         */
        getPropertyValue(sName) {
            const vValue = super.getPropertyValue(sName);
            switch (sName) {
                case "labelVisible":
                    // We only want to hide the label if the configuration is explicitly set to
                    // Absent/undefined values should be treated as "true" (visible).
                    return vValue !== false;
            }
            return vValue;
        }
    }
}
```

```

    };

    return MyWidget;
});

```

Reflect Configuration Changes

If changing a widget property in the editor of the Manage PODs 2.0 app also affects its appearance, the widget should override the `setProperty` method to ensure the widget's appearance matches the configured value.

This example assigns the label control to a private instance property so it can be referenced by other methods, then changes its visibility to match the new configuration value.

i Note

In this example the `getDefaultConfig` and `getPropertyValue` methods are shown. Other methods from previous examples are hidden for brevity, but are still present in the class.

Sample Code

```

sap.ui.define([
    "sap/m/Input",
    "sap/m/Label",
    "sap/m/Panel",
    "sap/dm/dme/pod2/propertyeditor/BooleanPropertyEditor",
    "sap/dm/dme/pod2/propertyeditor/PropertyCategory",
    "sap/dm/dme/pod2/widget/Widget",
    "sap/dm/dme/pod2/widget/metadata/WidgetProperty"
],
(
    Input,
    Label,
    Panel,
    BooleanPropertyEditor,
    PropertyCategory,
    Widget,
    WidgetProperty
) => {
    "use strict";

    class MyWidget extends Widget {
        /** @type {sap.m.Label} */
        #oLabel;

        /**
         * Create a panel to contain all the widget's UI elements.
         *
         * @override
         * @returns {sap.m.Panel}
         */
        _createView() {
            const oInput = new Input();
            this.#oLabel = new Label({
                labelFor: oInput,
                showColon: true,
                text: "My Input",
                visible: this.getPropertyValue("labelVisible")
            });

            return new Panel({
                headerText: "My Custom Panel Widget",
                content: [
                    this.#oLabel,
                    oInput
                ]
            });
        }
    }
}

```



```

    /**
     * Override to update the view preview.
     *
     * @override
     * @param {string} sName
     * @param {any} vValue
     */
    setPropertyValue(sName, vValue) {
        switch (sName) {
            case "labelVisible":
                this.#oLabel.setVisible(vValue);
            }
        super.setPropertyValue(sName, vValue);
    }
}

return MyWidget;
});

```

Add Widget Events

Create events for your widget.

Widgets may expose one or more configurable events which can be triggered by user input, such as a button being pressed or a list item being chosen. When a widget event occurs, a sequence of configured actions will be run.

Example

This example defines a new event to be configured with the `buttonPress` identifier and adds a new Button control to the `MyWidget` plugin. When the button is pressed, the event handler function:

1. Sets the value of the Input text box to a custom path in the POD Context so that other plugins can access it.
2. Calls the `_handleEvent` method which will instantiate and execute the configured action sequence for the `buttonPress` widget event.

i Note

In this example, methods that have not changed from previous examples are hidden for brevity, but are still present in the class.

≡ Sample Code

```

sap.ui.define([
    "sap/m/Button",
    "sap/m/Input",
    "sap/m/Label",
    "sap/m/Panel",
    "sap/dm/dme/pod2/context/PodContext",
    "sap/dm/dme/pod2/propertyeditor/BooleanPropertyEditor",
    "sap/dm/dme/pod2/propertyeditor/PropertyCategory",
    "sap/dm/dme/pod2/widget/Widget",
    "sap/dm/dme/pod2/widget/metadata/WidgetProperty"
],
(
    Button,
    Input,

```

```

Label,
Panel,
PodContext,
BooleanPropertyEditor,
PropertyCategory,
Widget,
WidgetProperty
) => {
    "use strict";

    class MyWidget extends Widget {
        /** @type {sap.m.Label} */
        #oLabel;

        /** @type {sap.m.Input} */
        #oInput;

        /**
         * Create a panel to contain all the widget's UI elements.
         *
         * @override
         * @returns {sap.m.Panel}
         */
        _createView() {
            this.#oInput = new Input();
            this.#oLabel = new Label({
                labelFor: oInput,
                showColon: true,
                text: "My Input",
                visible: this.getPropertyValue("labelVisible")
            });
            const oButton = new Button({
                text: "OK",
                press: this.#onButtonPress.bind(this)
            });

            return new Panel({
                headerText: "My Custom Panel Widget",
                content: [
                    this.#oLabel,
                    this.#oInput,
                    oButton
                ]
            });
        }

        /**
         * Set the input value to the POD Context and run any configured actions.
         *
         * @param {sap.m.Button$PressEvent} oEvent
         */
        #onButtonPress(oEvent) {
            // Set the current input value to a unique path in the POD Context so other plugins
            PodContext.set("/myCompany/myWidgetInput", this.#oInput.getValue());
        }
    }
}

```

```

        // Run any configured actions for the "buttonPress" event.
        this._handleEvent("buttonPress", oEvent)
    }

    /**
     * @override
     * @returns {Array<sap.dm.dme.pod2.widget.metadata.WidgetEvent>}
     */
    getEvents() {
        return [
            new WidgetEvent({
                id: "buttonPress",
                displayName: "Button Press",
                description: "Actions to run when the button is pressed"
            })
        ];
    }
}

return MyWidget;
});

```

Develop an Action

Learn how to develop your own action.

An action is a configurable procedure that occurs in response to user input. Actions are like reusable and configurable blocks of business logic. Actions may present a form or prompt to the user, but generally have a distinct start and end, and are not visibly part of a configurable page or dialog. A sequence of actions can be configured to run for each event supported by a widget.

Continue reading to learn how to develop your own action.

Create an Action Class

Create a custom JavaScript class for your action.

Context

When developing a new action, start by creating a JavaScript class that extends the `Action` base class. This page outlines the steps and naming conventions required to create an action class within SAP Digital Manufacturing.

Procedure

1. In the action folder of your project, create a new JavaScript file with the desired name for your action, for example `MyAction.js`.
2. Within the file, use [sap.ui.define](#) to create a new SAPUI5 module which imports the `Action` parent class exports a class which inherits from it.

☰ **Sample Code**

```

sap.ui.define([
    "sap/dm/dme/pod2/action/Action"
],
(
    Action
) => {
    "use strict";

    class MyAction extends Action {
    }

    return MyAction;
});

```

Add Action Metadata

Add metadata to your action class.

Context

Providing custom metadata is necessary to define how your action is presented in the interface of the editor in the [Manage PODs 2.0](#) app. This is done by implementing a set of static methods in your action class. These methods inform the editor about the action's name, icon, category, and optional help documentation.

Procedure

Add the following static methods to your action class definition:

- a. `getDisplayname()`: Specifies the name shown for the action in the [Add Action](#) dialog. This should return a localized string. Refer to [Internationalization](#) for localization guidelines.
- b. `getDescription()`: Specifies the tooltip shown for the action in the [Add Action](#) dialog. This should return a localized string. Refer to [Internationalization](#) for localization guidelines.

Sample Code

```

class MyAction extends Action {
    /**
     * Returns the display name shown in the action selection dialog.
     * @override
     * @returns {string}
     */
    static getDisplayname() {
        return "Sample Action";
    }

    /**

```

```

    * (Optional) Returns the display name shown in the action selection dialog.
    * @override
    * @returns {string}
    */
    static getDescription() {
        return "A sample action demonstrating the process of creating an extension.";
    }
}

```

Add Execute Method

Create an execute method in your action class.

Context

The execute method defined in your action class is invoked by the POD Runtime when the action is triggered. This method performs the main logic of the action and can be implemented as either a synchronous or asynchronous function.

→ Tip

Use an asynchronous execution when your action requires additional user interaction or a REST call.

The execute method receives an `ActionContext` object that provides contextual data such as the triggering widget and the SAPUI5 event object. The `ActionContext` also includes an abort function, which marks the action as aborted and stops the execution of any remaining actions configured for the event.

Procedure

Implement the execute method for your action in one of two ways:

- Synchronous execution: Define execute as a regular method. Use this approach for logic that completes immediately.

Sample Code

```

class MyAction extends Action {
    /**
     * @override
     * @param {sap.dm.dme.pod2.action.ActionContext} oActionContext
     */
    execute(oActionContext) {
        console.log("Hello World!");
    }
}

```

- Asynchronous execution: Define execute as an `async` method that returns a `Promise<void>`. Use this approach when calling external APIs, showing dialogs, or performing other logic which does not immediately run to completion.

Sample Code

```

class <class> extends Action {
  /**
   * @override
   * @param {sap.dm.dme.pod2.action.ActionContext} oActionContext
   * @returns {Promise<void>}
   */
  async execute(oActionContext) {
    ...
    const oResponse = await ApiClient.internal.processengine.start(oRequest);
    ...
  }
}

```

i Note

An asynchronous execute call can return a promise where the promise can be resolved later instead of using `async` or `await` syntax. This is primarily useful for asynchronous processes which cannot be easily awaited, for example waiting for a user to provide input through a dialog or other prompt.

≡, Sample Code

```

class MyAction extends Action {
  /**
   * Store the resolve function for the promise in a private property to be called later.
   * @type {Function}
   */
  #fnResolve;

  /**
   * @override
   * @param {sap.dm.dme.pod2.action.ActionContext} oActionContext
   * @returns {Promise<void>}
   */
  execute(oActionContext) {
    const oDialog = new Dialog({
      content: [
        new Text({ text: "Press the OK button to proceed" })
      ],
      beginButton: new Button({
        text: "OK",
        press: () => {
          // Resolve the pending Promise so the action sequence can continue.
          this.#fnResolve();
          oDialog.close();
        }
      }),
      afterClose: () => oDialog.destroy()
    });

    oDialog.open();

    // Create a Promise to represent the pending state of the dialog opened above,
    // then store its resolve function in a private property to be called once the dialog ha

```

```

        return new Promise((resolve) => {
            this.#fnResolve = resolve;
        });
    }
}

```

Add Action Properties

Define configurable properties for your action.

Context

Actions can include properties that allow you to customize how the action behaves. These properties are configured in the editor of the **Manage PODs 2.0** app using the **Edit Actions** dialog, and are stored in the POD configuration as part of the action definition. At runtime, your action retrieves these specified properties to control its behavior. If a property is not defined in the configuration, the action is responsible for handling the absence by applying a default value.

Procedure

Add a `getProperties()` method to your action class and return an array of `ActionProperty` objects. Each object describes one configurable property:

- a. `displayName`: The text to display in the **Properties** panel for this property.
- b. `description`: The tooltip to display when the user mouses over the property to give more detailed information on what the property does.
- c. `propertyEditor`: An instance of `sap.dm.dme.pod2.propertyeditor.PropertyEditor` that provides a control to display and modify the action property. Typically you would use an existing property editor class. For more information, see [Property Editors](#).

Sample Code

```

getProperties() {
    return [
        new ActionProperty({
            displayName: "Create SFC on Start",
            description: "Automatically create an SFC on start.",
            propertyEditor: new BooleanPropertyEditor(this, "sfcVisible")
        })
    ];
}

```

Note

To support internationalization, use the `getI18nText` method to retrieve localized strings for `displayName` and `description`. Refer to [Internationalization](#) for details.

Example

This example adds a configurable property to specify the text to be displayed in the dialog created in the previous example.

Sample Code

```
class MyAction extends Action {
  /**
   * Store the resolve function for the promise in a private property to be called later.
   * @type {Function}
   */
  #fnResolve;

  /**
   * @override
   * @param {sap.dm.dme.pod2.action.ActionContext} oActionContext
   * @returns {Promise<void>}
   */
  execute(oActionContext) {
    const oDialog = new Dialog({
      content: [
        new Text({
          text: this.getPropertyValue("dialogMessage") || "Press the OK button to proc
        })
      ],
      beginButton: new Button({
        text: "OK",
        press: () => {
          // Resolve the pending Promise so the action sequence can continue.
          this.#fnResolve();
          oDialog.close();
        }
      }),
      afterClose: () => oDialog.destroy()
    });

    oDialog.open();

    // Create a Promise to represent the pending state of the dialog opened above,
    // then store its resolve function in a private property to be called once the dialog ha
    return new Promise((resolve) => {
      this.#fnResolve = resolve;
    });
  }

  getProperties() {
    return [
      new ActionProperty({
        displayName: "Dialog Message",
        description: "Specify the text to be displayed in the dialog.",
        propertyEditor: new StringPropertyEditor(this, "dialogMessage")
      })
    ];
  }
}
```

Next Steps

To retrieve the value of a property you've configured, call `getPropertyValue`:

```
/**
 * @override
 * @param {sap.dm.dme.pod2.action.ActionConfig} oActionConfig
 */
constructor(oConfig) {
  super(oConfig);

  if (this.getPropertyValue("sfcVisible")) {
    ...
  }
}
```


Extend a Core Plugin

You can extend a core plugin to make minor alterations.

Caution

Plugin extensions are ideal for minor modifications to core plugins, like adding buttons, fields, or changing labels and colors. They are not meant for major alterations. For significant changes, we recommend developing your custom plugin from scratch.

Core plugins utilize a class inheritance model, enabling you to easily extend core widgets and actions by building on existing functionality. You can extend a core plugin by defining a child class that inherits from the parent class, giving it access to core plugin functions and properties. By overriding specific methods, you can customize the plugin while maintaining compatibility with any future updates.

Note

Changes made to core plugins aren't automatically reflected in existing PODs. Once you've created a custom POD plugin which extends a core plugin, you should also replace instances of that core plugin in PODs that should use your custom behavior.

Note

Only some classes and methods can be extended or overridden:

- Classes marked with `This class may be extended by custom subclasses` can be extended.
- Methods marked with `This method may be overridden by custom subclasses` can be overridden.

For additional information, refer to the [POD 2.0 JSDoc](#) ➡

Property Editors

Learn about plugin property editors.

A property editor is an object that is associated with a specific plugin property, such as a `WidgetProperty` or `ActionProperty`. The property editor is responsible for rendering the appropriate UI5 control based on the property's data type and constraints.

We provide a number of standard property editors for you to use, or you can develop your own custom property editor. For additional information, refer to the [POD 2.0 JSDoc](#) ➡

Custom Property Editors

Create a custom property editor for your plugin.

Custom property editors can be created using a subclass of `sap.dm.dme.pod2.propertyeditor.PropertyEditor`. Custom property editors can create any type of control to display in the **Properties** panel in the editor of the **Manage PODs 2.0** app.

Example

If a widget has a property that requires complex configuration and the value stored is a complex JSON object, then a custom property editor can be created that displays a button. Choosing this button launches a dialog to configure the property value

and store it as a JSON object.

≡ Sample Code

```
sap.ui.define([
    "sap/dm/dme/pod2/propertyeditor/PropertyEditor"
], (
    PropertyEditor
) => {
    "use strict";

    /**
     * @alias sap.dm.dme.pod2.propertyeditor.MyCustomPropertyEditor
     * @extends sap.dm.dme.pod2.propertyeditor.PropertyEditor
     */
    class MyCustomPropertyEditor extends PropertyEditor {
        #oControl;

        /**
         * @param {sap.dm.dme.pod2.propertyeditor.$PropertyAccessor} oPropertyAccessor The widget
         * object which satisfies the $PropertyAccessor interface.
         * @param {string} sProperty The property name.
         * @param {Object} [oOptions]
         * @param {boolean} [oOptions.exampleOption] An example of an option.
         */
        constructor(oObject, sProperty, oOptions = {}) {
            super(oObject, sProperty);
            this.#oControl = new ...;
        }

        /**
         * @returns {sap.ui.core.Control}
         */
        getControl() {
            return this.#oControl;
        }
    }

    return MyCustomPropertyEditor;
});
```

Internationalization

Learn how to enable and use translated text in custom plugins.

To enable translations, plugins can override the static `getI18nModel()` method and return an instance of `sap.dm.dme.pod2.model.I18nResourceModel`. It is recommended to define the model as a static property so it loads when the class is initialized. For plugins with limited or no translations, use the `supportedLocales` configuration to avoid unnecessary errors when the framework attempts to load unsupported locales. If only one default resource file exists, set `supportedLocales: [""]`.

The POD Runtime determines the language in the following order:

- The sap-language URL parameter
- The user's Fiori Launchpad language setting
- The browser or system language

The model returned by `getI18nModel()` can include the `processBundleAvailable` setting to support process industry terminology. When enabled, the framework attempts to load a second resource bundle with a `pi_` prefix, such as `pi_i18n.properties`, in addition to the base bundle. This override is applied only if the plant is configured for process industry. If not, the additional bundle is ignored.

→ Tip

As an alternative to overriding `getI18nModel()`, a plugin can override `getI18nModelSettings()` to return the configuration settings for the `I18nResourceModel` constructor. The model will then be created automatically. This approach is simpler, but may have a performance impact since the model is not created at class load time.

To retrieve translated text, use `this.getI18nText(sKey)`. For texts with placeholders, pass additional values: `this.getI18nText(sKey, sValue1, sValue2)`.

To enable data binding for localized text, assign the i18n model to your view. For example, in `_createView()`:

≡ Sample Code

```
oView.setModel(this.getI18nModel(), "i18n-internal");
```

Logging

Enable logging in your plugin.

Context

Implementing logging in your custom plugin can provide structured support for debugging, tracing user actions, and monitoring runtime behavior. Logs are printed to the browser's developer console and can include colored output and stack traces for better readability. You can control logging behavior using configurable log levels.

Best Practices for Logging

- Write log messages in English using proper grammar
- Log all caught exceptions
- Log all user interactions
- Include relevant context, such as SFC numbers or object IDs

Procedure

1. Import the `Logger` module into your class using `sap.ui.define`.

≡ Sample Code

```

sap.ui.define([
    "sap/dm/dme/pod/Logger",
],
/**
 * @param {typeof sap.dm.dme.pod2.Logger} Logger
 */
(
    Logger
) => {
    "use strict";

```

2. Get a logger instance, and use the fully qualified class name as the identifier when calling `Logger.getLogger()`.

Sample Code

```
const oLog = Logger.getLogger("sap.dm.dme.pod2.MyClass");
```

3. Write your log message with the appropriate level.

i Note

The Logger has multiple levels of logging: FATAL, ERROR, WARN, INFO, DEBUG, TRACE with the default level of INFO.

Sample Code

```
oLog.info("This is a simple log message!");
```

4. Set the default log level or override the log level for a specific logger:

- o To set the default log level:

Sample Code

```
Logger.setDefaultLevel(Logger.Level.DEBUG);
```

- o To override the log level:

Sample Code

```
oLog.setLevel(Logger.Level.TRACE);
```

Packaging and Uploading Custom Plugins

Learn how to package and upload an extension so that your plugins are available for use with SAP Digital Manufacturing.

Context

The **Manage PODs 2.0** app uses your `extension.json` file to determine which custom plugins are available in the editor. These plugins are automatically available in the **Plugins** tab of the left panel in the **Manage PODs 2.0** editor with a blue row highlight indicator, as well as the **Design Production Processes** app.

Procedure

1. Add the following sample code to the `extension.json` file for your extension, where:

- `myCompany/extension` is your namespace prefix
- `MyWidget` is the name of your widget JavaScript file (if applicable)
- `MyAction` is the name of your action JavaScript file (if applicable)

i Note

We recommend that you give each extension a type string that is the same as its module path, using dot-separated notation instead of a slash-separated path. This is not a requirement, but ensures descriptive and unique type strings.

i Note

Either the `widgets` or `actions` array can be omitted if the extension only includes one type of plugin.

Sample Code

```
{
  "widgets": [
    {
      "modulePath": "myCompany/extension/widget/MyWidget",
      "type": "myCompany.extension.widget.MyWidget"
    }
  ],
  "actions": [
    {
      "modulePath": "myCompany/extension/action/MyAction",
      "type": "myCompany.extension.action.MyAction"
    }
  ]
}
```

2. Package the extension by compressing the contents of your project to a ZIP, TAR, or TGZ file.

i Note

Make sure to compress your extension so that the `extension.json` file is at the root.

3. On your SAP Digital Manufacturing Fiori Launchpad, navigate to the **Extensions** tab of the **Manage PODs 2.0** app, choose **Create** and complete the following fields in the dialog that opens:

- **Name** (Required): It's important to know that you can't change the name once you've created the widget or action.
- **Description** (Optional): Provide a description for the plugin.
- **Namespace** (Required): A valid namespace consists of alphanumeric characters, which can be separated by periods, underscores, or slashes. Each segment between these delimiters can contain only alphanumeric characters, and the total length of the namespace can be up to 100 characters. Ensure that this field matches the namespace in your `extension.json` file and references in source code files.

Example

The namespace for the sample code above is `myCompany/extension`.

- **Source Code** (Required): Provide your compressed extension file. A valid input is a ZIP, TAR, or TGZ file that is under 5MB and contains all the source code files for your extension.

4. Choose **Create**.

Results

You've packaged and uploaded your extension to the [Manage PODs 2.0](#) app. Your plugins are now visible in the [Plugins](#) tab of the left panel of the editor in the [Manage PODs 2.0](#) app.

POD Plugin Migration Guide

Learn how to migrate your existing plugins to the [Manage PODs 2.0](#) app.

Your existing POD plugins can be migrated for use in the [Manage PODs 2.0](#) app. There are two main methods to achieve this:

- **Creating a New Plugin:** Build a new widget from the ground up and selectively copy or refactor your original business logic. Though this approach requires a greater initial effort, it results in a more maintainable and optimized plugin that aligns closely with the [Manage PODs 2.0](#) app.
- **Wrapper-Based Migration:** Wrap your existing plugin's UI5 component in a new `ComponentWidget` class. While quicker to implement, this approach can lead to reduced maintainability and potential inefficiencies.

i Note

We recommend creating a new plugin. To do this, simply create a new directory for your widget (outside of your usual POD plugin folder), and refer to [POD 2.0 Plugin Developer's Guide](#). When creating your new plugin, you have the benefit of using your original plugin as a reference, and can keep some of the existing code and business logic.

Before migrating your plugin to the [Manage PODs 2.0](#) app, consider the following:

- Whether there is value in splitting your existing plugin into 2 or more new plugins.

❖ Example

A plugin with both a table and an edit form can be split in two. This allows you to use the edit form independently, show the form in a dialog, or extend functionality with custom buttons.

- Whether it's necessary to create an equivalent widget that consists of a single UI5 control with some logic.

❖ Example

If an icon with color was used to show SFC status in your original plugin, you can wrap this UI5 icon in a standalone widget that determines status and applies color automatically.

Wrapper-Based Migration

Migrate your existing plugin using a wrapper class.

Context

You can migrate an existing plugin to the [Manage PODs 2.0](#) app by wrapping it inside a `ComponentWidget` class. This allows you to preserve most of your original structure while enabling compatibility with the new app. Follow the steps below to complete a wrapper-based migration.

Procedure

1. Create a new folder for your plugin wrapper outside of your existing POD plugin folder. This helps maintain a clear separation between your original code and the new wrapper structure.

2. Copy your original plugin files into the new directory. This typically includes your UI5 Component and any associated resources.

i Note

If your plugin relies on shared libraries or other resources that are not relevant to **Manage PODs 2.0**, copy them over only when needed. Avoid copying everything up front to ensure that you only include relevant, actively used code.

3. Create a new class that extends the `sap.dm.dme.pod2.widget.ComponentWidget` base class. This wrapper class allows the legacy UI5 component to function within the new plugin system.
4. Review your existing plugin code and identify any functions or methods that are no longer supported, and replace them with updated equivalents. For details, refer to [Legacy Function Equivalents](#).
5. Ensure your property editor is integrated into the new wrapper. If your original plugin used a container-style property editor, adapt it to support individual controls that yield values to be persisted to the configuration.
6. If your plugin uses custom models or styles defined in the `manifest.json` file, make sure these are available to the wrapper. This may involve modifying how resources are loaded so that your plugin can access them as needed.
7. Update any unit tests that reference old plugin paths or structures. Ensure that your tests reflect the new wrapper-based architecture and validate behavior in the context of **Manage PODs 2.0**.

Legacy Function Equivalents

The following table lists legacy POD plugin functions and their new equivalents.

Purpose	Legacy Function	New Equivalent
Get the current plant	<code>PlantSettings.getCurrentPlant()</code>	<code>PodContext.getPlant()</code>
Get the plant time zone	<code>PlantSettings.getTimeZone()</code>	<code>PodContext.getPlantTimeZone()</code>
Get the industry type for the plant	<code>PlantSettings.getIndustryType()</code>	<code>PodContext.getIndustryType()</code> <code>PodContext.isDiscreteIndustry()</code> <code>PodContext.isProcessIndustry()</code>
Get POD configuration values	<code>this.getConfiguration()[propertyName]</code>	<code>this.getPropertyValue(propertyName)</code> <code>this.getConfig().properties[propertyName]</code>
Get any reference to shared data/state	<code>this.getPodSelectionModel().get__()</code> <code>this.getPodSelectionModel().set__(value)</code>	<code>PodContext.get__()</code> <code>PodContext.set__(value)</code> For custom properties: <code>PodContext.get(key, value)</code> <code>PodContext.set(key, value)</code> i Note For a list of core entities and their respective keys, refer POD 2.0 JSDoc
Clear POD selections	<code>this.getPodSelectionModel().clearSelections()</code>	<code>PodContext.setSelectedWorkListItems([])</code>

Purpose	Legacy Function	New Equivalent
Modify subscriptions to shared data/state changes	<pre> this.subscribe("PodSelectionChangeEvent", this._handlePodSelectionChange.bind(this)) this.unsubscribe("PodSelectionChangeEvent", this) </pre>	<pre> PodContext.subscribe(key, handler, this) PodContext.unsubscribe(key, handler, this) PodContext.unsubscribeAll(this) </pre> i Note For a list of core entities and their respective keys, refer POD 2.0 JSDoc i Note It is best to include a check in your handler function to ensure that the data is not null. For example, if the filter work center is changed to a new object, if the filter work center data, the following condition can avoid performing unnecessary calculations: ⌘ Sample Code <pre> if (oNewWorkCenter?.workCenter === oPreviousWorkCenter) { return; } </pre> Additionally, ensuring that the widget does not refresh unnecessarily can help to reduce unnecessary traffic: ⌘ Sample Code <pre> if (!this.isVisible()) { return; } </pre>
Get translated text	<pre> this.getI18nText(key) this.getI18nText(key, [arg1, arg2]) </pre>	<pre> this.getI18nText(key) this.getI18nText(key, arg1, arg2) </pre> i Note If the widget/action provides its own translation model, the widget's model will be checked first. If no override model is provided or no translation is found, the widget will use the PodContext.getI18nText.
Be informed of POD websocket notifications	Implement <code>getNotificationMessageHandler</code> in the <code>PluginViewController</code> subclass, in combination with a POD configuration setting to enable certain topics and subscriptions	⌘ Sample Code <pre> PodNotificationWebSocket.subscribe({ eventType, onMessage, filter, description, }); </pre>

Purpose	Legacy Function	New Equivalent
Get the POD type	this.isPodType(PodType.WorkCenter)	i Note The concept of POD types has been deprecated in Manufacturing. The type of action performed can be inferred based on the available data. When a specific item is selected, for example: ≡ Sample Code <pre>if (PodContext.getLastSelectedWorkListItem() != null) { // This is an order-based workflow }</pre> Many functions which previously depended on the POD actions, allowing the behavior to be configured as-needed assumptions about how the POD will be used.